Solution: Number of Islands

Problem Statement

Given a 2D array (i.e., a matrix) containing only 1s (land) and 0s (water), count the number of islands in it.

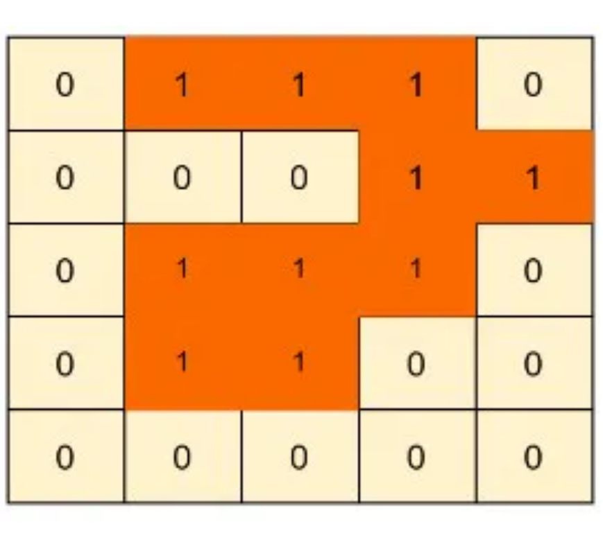
An **island** is a connected set of 1s (land) and is surrounded by either an edge or 0s (water). Each cell is considered connected to other cells horizontally or vertically (not diagonally).

Example 1

Input: matrix =

0	1	1	1	0
0	0	0	1	1
0	1	1	1	0
0	1	1	0	0
0	0	0	0	0

Output: 1
Explanation: The matrix has only one island. See the highlighted cells below.



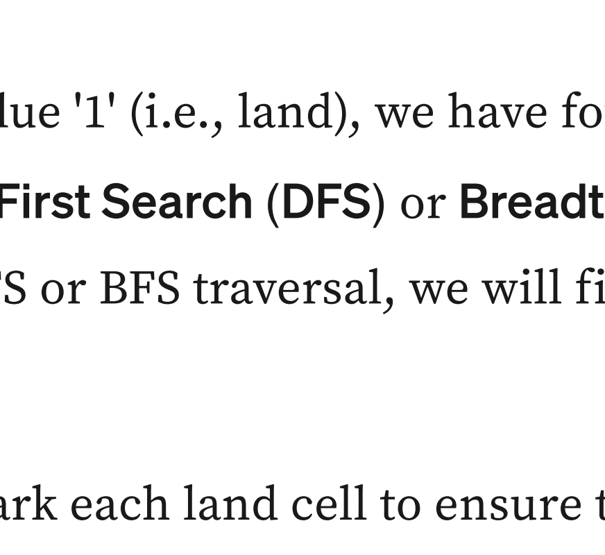
0	1	1	1	0
0	0	0	1	1
0	1	1	1	0
0	1	1	0	0
0	0	0	0	0

Example 2

Input: matrix =

1	1	1	0	0
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	1	0	0

Output: 3
Explanation: The matrix has three islands. See the highlighted cells below.



1	1	1	0	0
0	1	0	0	1
0	0	1	1	0
0	0	1	0	0
0	0	1	0	0

Solution

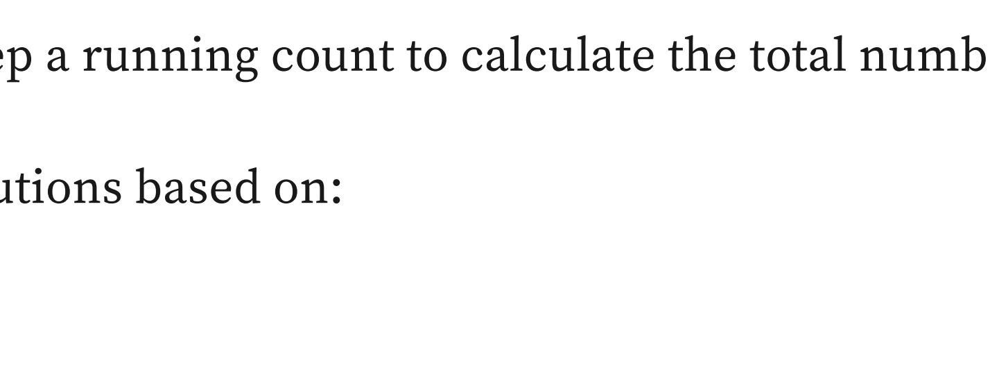
We can traverse the matrix linearly to find islands.

Whenever we find a cell with the value '1' (i.e., land), we have found an island. Using that cell as the root node, we will perform a **Depth First Search** (**DFS**) or **Breadth First Search** (**BFS**) to find all of its connected land cells. During our DFS or BFS traversal, we will find and mark all the horizontally and vertically connected land cells.

We need to have a mechanism to mark each land cell to ensure that each land cell is visited only once. To mark a cell visited, we have two options:

1. We can update the given input matrix. Whenever we see a '1', we will make it '0'.
2. A separate boolean matrix can be used to record whether or not each cell has been visited.

Following is the DFS or BFS traversal of the example-2 mentioned above:



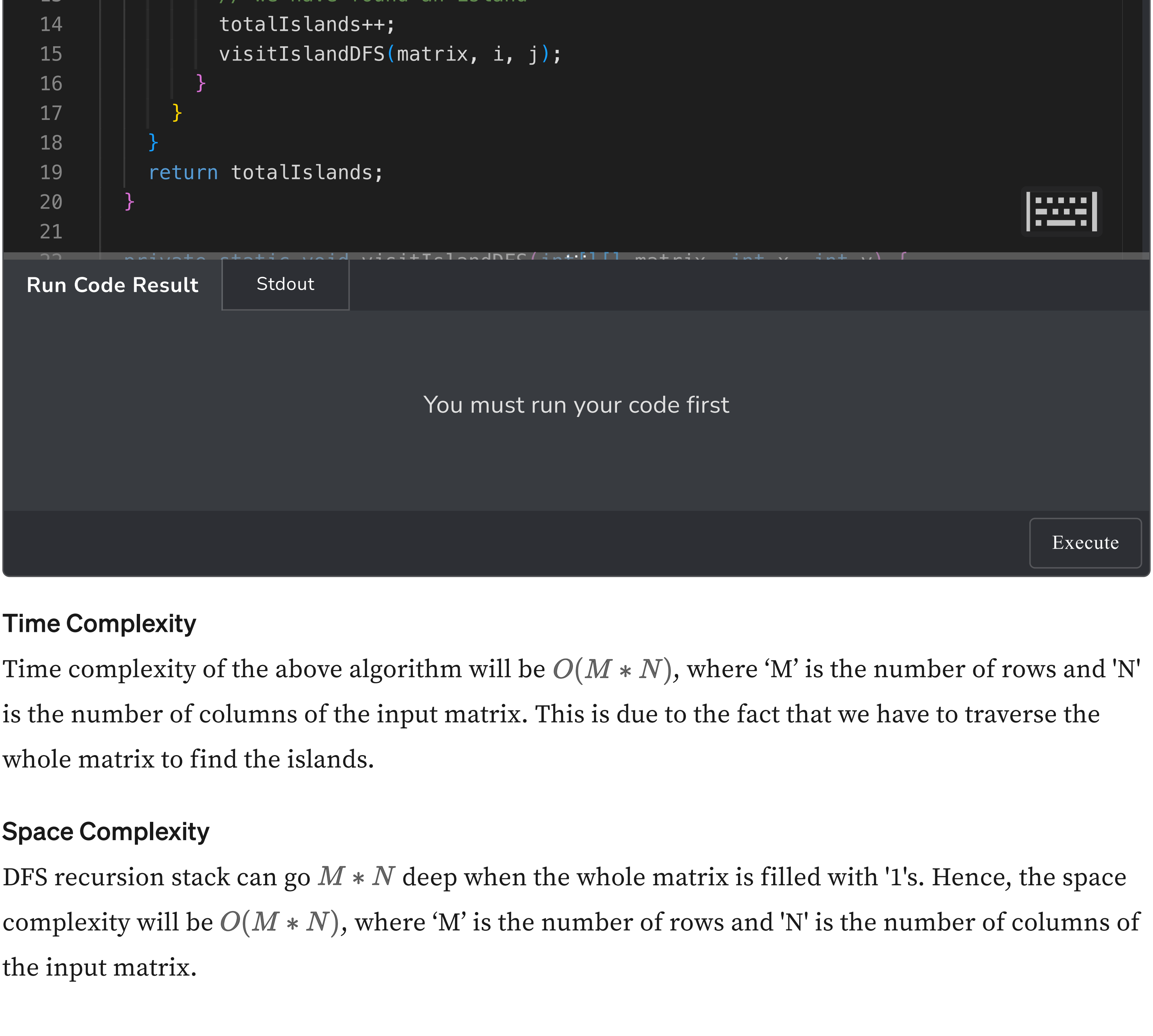
By following the above algorithm, every time DFS or BFS is triggered, we are sure that we have found an island. We will keep a running count to calculate the total number of islands.

Below, we will see three solutions based on:

1. DFS
2. BFS
3. BFS with visited matrix

Code (DFS)

Here is what our DFS algorithm will look like. We will update the input matrix to mark cells visited.



```java
import java.util.*;

class Solution {
  public int countIslands(int[][] matrix) {

    int rows = matrix.length;
    int cols = matrix[0].length;
    int totalIslands = 0;

    for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == 1) { // only if the cell is a land
          // we have found an island
          totalIslands++;
          visitIslandDFS(matrix, i, j);
        }
      }
    }
    return totalIslands;
  }

```

You must run your code first

Time Complexity

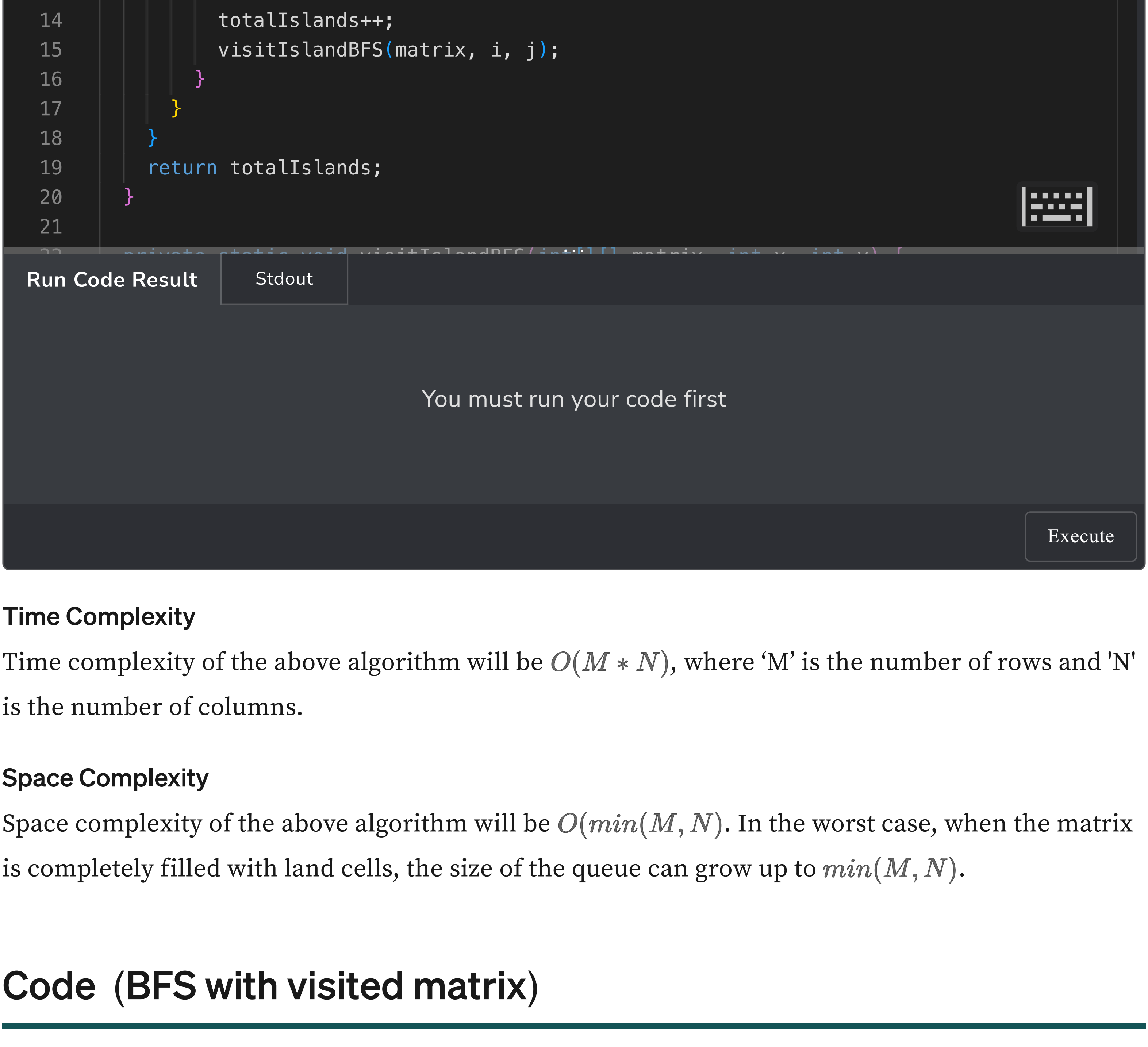
Time complexity of the above algorithm will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that we have to traverse the whole matrix to find the islands.

Space Complexity

DFS recursion stack can go $M * N$ deep when the whole matrix is filled with '1's. Hence, the space complexity will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix.

Code (BFS)

Here is what our BFS algorithm will look like. We will update the input matrix to mark cells visited.



```java
import java.util.*;

class Solution {
  public static int countIslands(int[][] matrix) {

    int rows = matrix.length;
    int cols = matrix[0].length;
    int totalIslands = 0;

    for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == 1) { // only if the cell is a land
          // we have found an island
          totalIslands++;
          visitIslandBFS(matrix, i, j);
        }
      }
    }
    return totalIslands;
  }

```

You must run your code first

Time Complexity

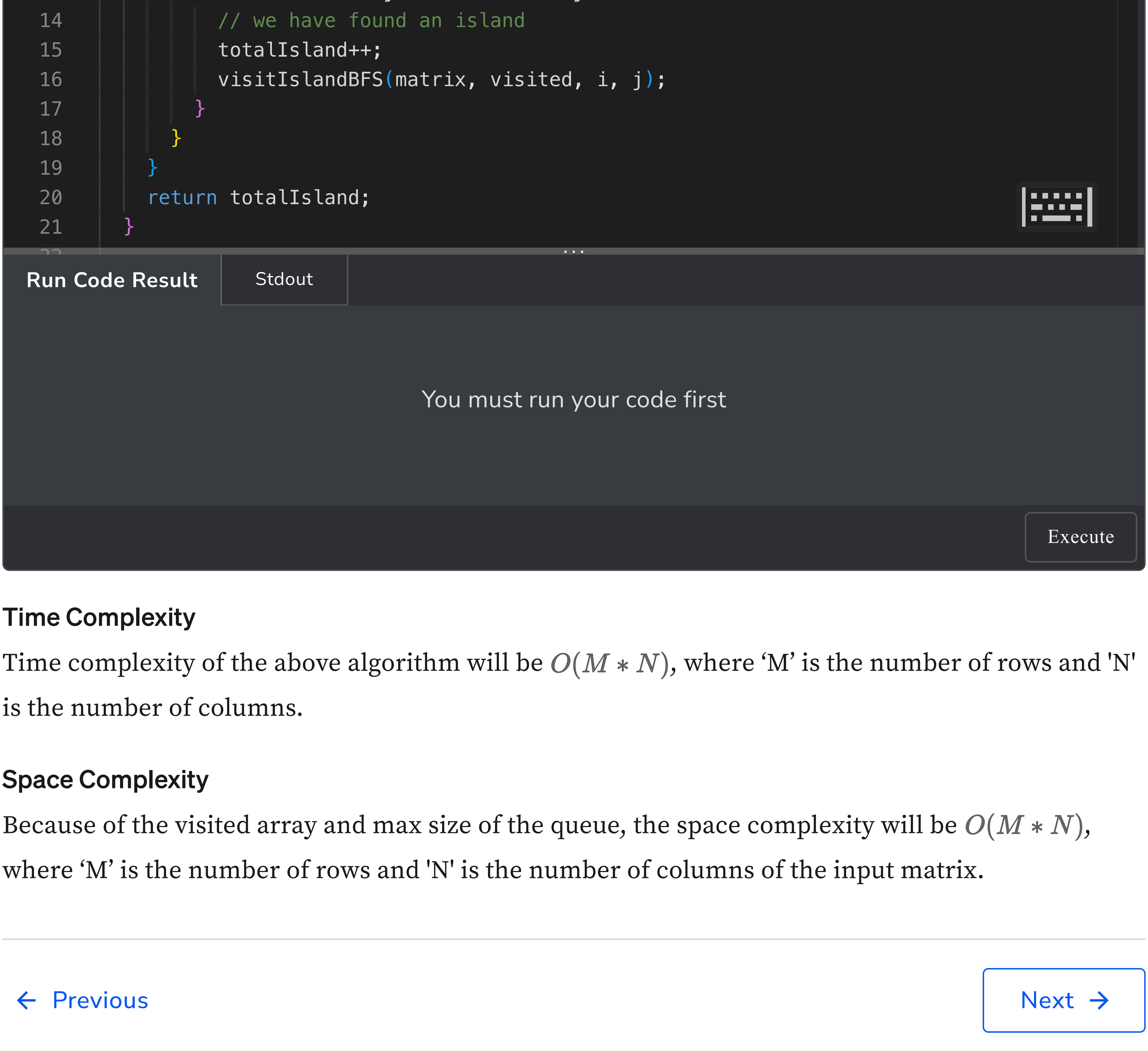
Time complexity of the above algorithm will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns.

Space Complexity

Space complexity of the above algorithm will be $O(min(M, N))$. In the worst case, when the matrix is completely filled with land cells, the size of the queue can grow up to $min(M, N)$.

Code (BFS with visited matrix)

Here is what our BFS algorithm will look like. We will keep a separate boolean matrix to record whether or not each cell has been visited.



```java
import java.util.*;

class Solution {
  public static int countIslands(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    boolean[][] visited = new boolean[rows][cols];
    int totalIsland = 0;

    for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++) {
        // if the cell has not been visited before and is a land
        if (!visited[i][j] && matrix[i][j] == 1) {
          // we have found an island
          totalIsland++;
          visitIslandBFS(matrix, visited, i, j);
        }
      }
    }
    return totalIsland;
  }
```

You must run your code first

Time Complexity

Time complexity of the above algorithm will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns.

Space Complexity

Because of the visited array and max size of the queue, the space complexity will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Number Of Islands (Easy)
Next →
Biggest Island (Easy)
☐ Mark as Completed